PROJECT CONCEPT BRIEF | SOFTWARE + AI/ML

TechPulse AI

Autonomous Technical Intelligence, Classification & Impact Prediction Platform

The core idea

A web platform that continuously collects technical papers and open-source project signals, classifies them, uses LLMs to turn them into evidence-backed insights, and applies supervised machine learning to estimate future impact or adoption.

From information overload to useful signals



Project objective

Help students, developers, and researchers find what matters in a fast-moving technical ecosystem - and understand why it matters, what may happen next, and what they could explore or build.

Recommended MVP: arXiv + GitHub, 5-8 technical domains, structured LLM analysis, one well-defined prediction target, and a usable dashboard. Add advanced agents and knowledge graphs after the core pipeline works.

1. Project overview

The problem is not a lack of technical information. It is the difficulty of filtering thousands of new papers, repositories, articles, and datasets to identify useful developments. TechPulse AI is designed as a continuous intelligence pipeline rather than a one-off chatbot.

What the platform should answer

- What is new? Discover relevant recent papers, repositories, and technical updates.
- What is it about? Assign a domain and explain the main contribution in plain language.
- Why does it matter? Surface evidence such as results, activity, growth, relevance, and limitations.
- What may happen next? Predict future citations, repository growth, impact category, or domain-level trends.
- What should I do next? Recommend items to read, technologies to investigate, or research gaps to explore.

Inputs and outputs

Input family	Examples	Platform output
Research content	Paper title, abstract, PDF, authors, date, categories	Domain, grounded summary, key claims, limitations, links
Open-source signals	Repository description, README, stars, forks, commits, releases	Project category, activity indicators, growth prediction
Time-series signals	Monthly paper counts, citation growth, stars gained, topic mentions	Emerging/rising/stable/declining trend estimates
User context	Selected interests, preferred domains, saved items	Ranked feed, relevant search results, learning/research suggestions

> **A crucial distinction**
> The LLM explains and synthesizes content. The supervised ML model learns from historical examples with defined labels. They are complementary components, not substitutes for one another.

Recommended first scope

Begin with arXiv papers and GitHub repositories in AI/ML, NLP, computer vision, generative AI, cybersecurity, robotics, data science, and systems. This is broad enough to demonstrate the idea but small enough for a student team to implement.

2. Product features that make it interesting

Feature	User experience	How it works
Signal score	Quickly separates high-value items from low-priority noise.	Combines validated predictive signals with transparent ranking rules; shows the evidence behind the rank.
Technology radar	Shows emerging, rising, established, and declining topics.	Aggregates monthly topic volume, repository activity, and other time-aware indicators.
Paper-to-repository links	Connects a research idea to code that implements it.	Matches titles, authors, URLs, README references, identifiers, and semantic similarity; confirms links when possible.
Research opportunity detector	Suggests possible next questions or limitations to investigate.	Compares limitations and methods across related papers; LLM proposes candidate gaps with supporting sources for human review.
Evidence-grounded research assistant	Answers questions over the collected corpus with links to source material.	Retrieves relevant chunks through vector/metadata search, then asks the LLM to synthesize only from retrieved evidence.
Prediction vs. reality	Shows how earlier impact forecasts performed.	Stores predictions with timestamps and compares them with outcomes observed after a fixed time horizon.
Hype vs. adoption	Flags topics with attention but weak implementation activity, or vice versa.	Compares research/mention growth against repository growth and other adoption proxies.

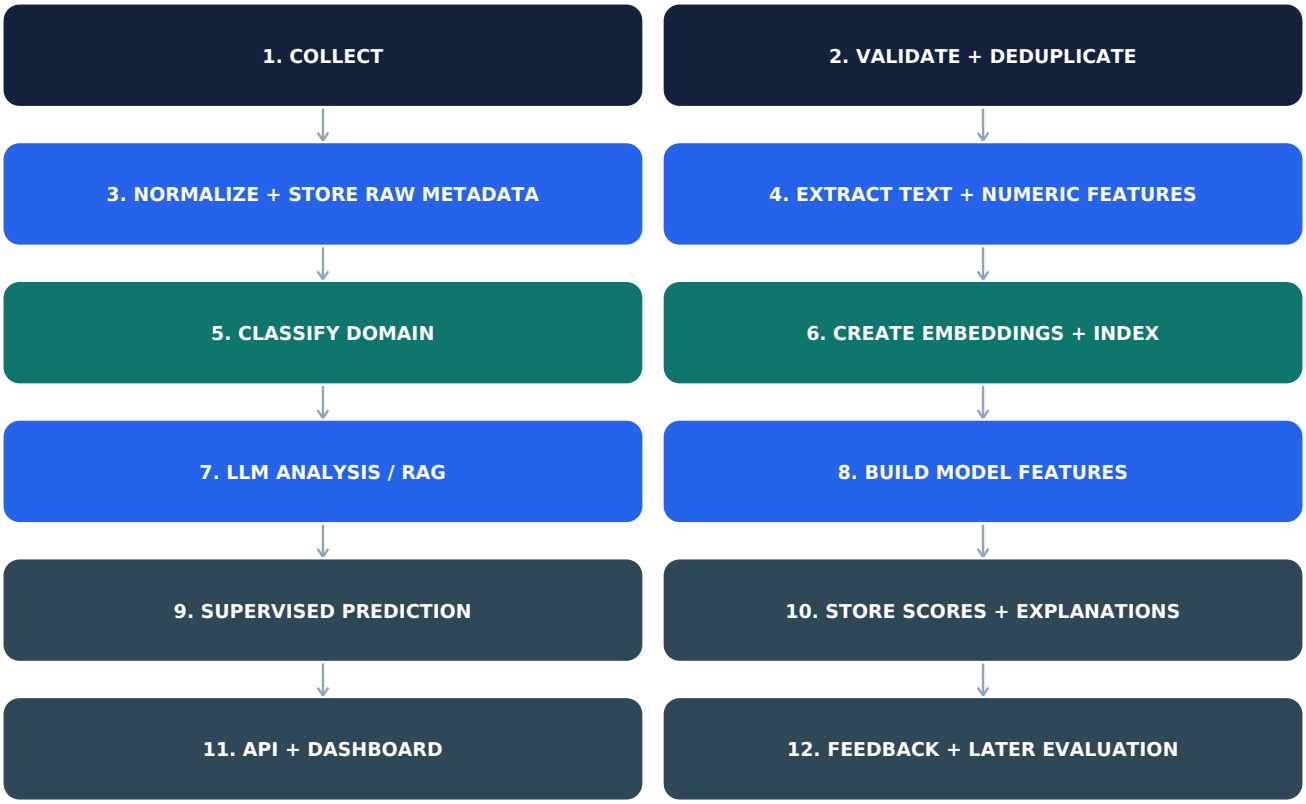
Prioritize the features

- Must-have: autonomous ingestion, domain classification, LLM summary/insights, a credible prediction baseline, interactive exploration dashboard.
- Strong differentiators: explainable signal score, evidence-grounded search, topic trend view, prediction history.
- Stretch goals: multi-agent critique, research-gap graph, paper-to-repository knowledge graph, personalized learning roadmaps.

> **Design principle**
> Every important score should have a visible explanation and links to its evidence. Avoid displaying a precise number, such as 91/100, as if it were an objective truth when it is only a heuristic or an unvalidated estimate.

3. End-to-end workflow

The system runs as a repeatable pipeline. It ingests new records, normalizes them, enriches them with AI analysis and features, predicts outcomes, stores results, and exposes them through the web application.



Workflow, in practical terms

01 | Discover - Scheduled jobs call the arXiv API, GitHub API, or approved RSS feeds. Fetch only records newer than the last successful run where possible.

02 | Normalize - Convert varied records into a common schema. Validate required fields and remove duplicates by stable IDs, canonical URLs, or normalized titles.

03 | Enrich - Extract paper text or README content, calculate text embeddings, assign a domain, and request structured LLM analysis.

04 | Predict - Build a feature vector using information available at the prediction timestamp. Apply the trained task-specific model.

05 | Publish - Store the item, sources, model version, feature timestamp, outputs, and prediction explanation. The frontend reads through an API.

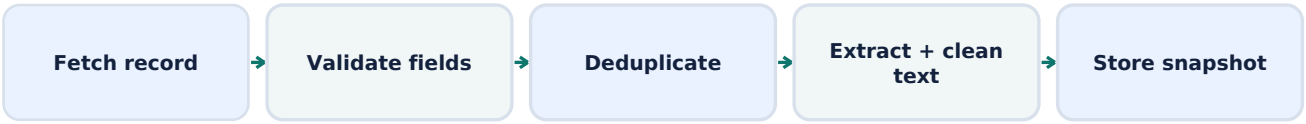
06 | Learn from outcomes - After the chosen horizon, collect actual citations or repository growth, compare against the forecast, and use newly matured records for retraining.

4. Data ingestion, processing, and storage

Start with two sources. Add more only when the ingestion, deduplication, and evaluation loop is reliable.

Source	Useful fields	Notes
arXiv	Paper ID, title, abstract, authors, categories, submitted/updated dates, PDF URL	Good for discovery and subject classification. Citation counts may require a separate source or delayed enrichment.
GitHub	Repository ID, name, description, topics, language, stars, forks, issues, commits/releases, created/updated dates	Collect time-stamped snapshots; current stars alone do not reveal growth. Respect API rate limits and terms.
Optional RSS feeds	Article title, publication date, URL, publisher, text/summary	Use a small allowlist of reliable sources and retain source attribution.

Processing sequence



Suggested normalized schema

Field	Purpose
item_id, source, source_url	Stable identity, data provenance, and a link to the source.
title, description, full_text / abstract	Text for classification, search, summarization, and feature extraction.
published_at, fetched_at, snapshot_at	Distinguishes when content was released, collected, and when measurements were observed.
domain, domain_confidence, tags	Predicted categories and their confidence/metadata.
numeric_features	Citations, stars, forks, activity counts, age, or other source-supported measures.
llm_analysis, evidence_refs	Structured summary, contributions, limitations, applications, and citations to exact evidence.
prediction, model_version, predicted_at	Output of the model and enough information to reproduce/evaluate it later.

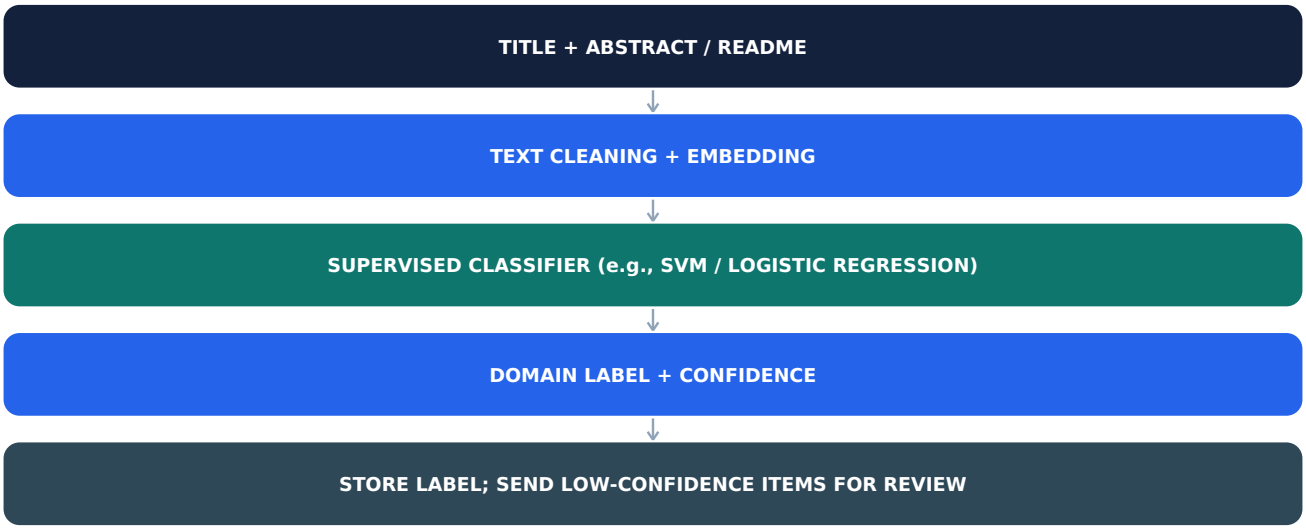
Storage recommendation

Use PostgreSQL for canonical records, time-stamped snapshots, prediction history, and user data. Use FAISS or another vector index for semantic retrieval. Keep references from every vector chunk back to its original source and section.

5. Classification, LLM analysis, and agents

Treat classification, content synthesis, and conversational retrieval as three separate jobs. This improves testing and makes errors easier to diagnose.

A. Domain classification



Create a labelled dataset across a manageable set of domains. A SentenceTransformer embedding can represent the text; train an SVM, logistic regression, or another classifier on labelled examples. Compare it with an LLM-based baseline. Use macro-F1 as well as accuracy if classes are imbalanced.

B. LLM insight extraction

- Send the title, abstract or selected text, metadata, and clear instructions to the LLM.
- Request structured fields: concise summary, claimed contribution, methodology, reported results, limitations, applications, and candidate research questions.
- Require the model to distinguish author-reported facts from its own interpretation. Keep page/section or URL references for important claims.
- Validate the output schema and handle invalid JSON, missing content, and model timeouts. Never let LLM-generated scores silently become ground-truth labels.

C. RAG research assistant



Embeddings and a vector index help find semantically related passages; metadata filters narrow the search by date, domain, source, or predicted impact. The generated answer should cite the retrieved items so users can verify it.

D. Multi-agent workflow (optional)

Use agents only where roles are distinct: a Research Analyst extracts evidence, an Adoption Analyst inspects repository signals, a Critic checks missing evidence and overclaims, and a Synthesizer prepares a final view. Agents should share sources and follow a bounded workflow; more agents do not automatically mean better answers.

6. How the supervised ML model will work

This is the core predictive component. First define a measurable future outcome. Then create historical examples where all input features were available at a known point in time and the future outcome is now observable.

Step 1 - Define the prediction target

Prediction task	Example target label	Model type
Paper impact	Number of citations gained in the next 180 days, or top-quintile impact within comparable field/age groups	Regression or binary/multiclass classification
Repository growth	Stars gained over the next 90 days, or top-quintile growth among comparable repositories	Regression or classification
Domain trend	Next-month / next-quarter topic volume or trend class from historical monthly counts	Time-series forecast or trend classification

Recommended MVP target

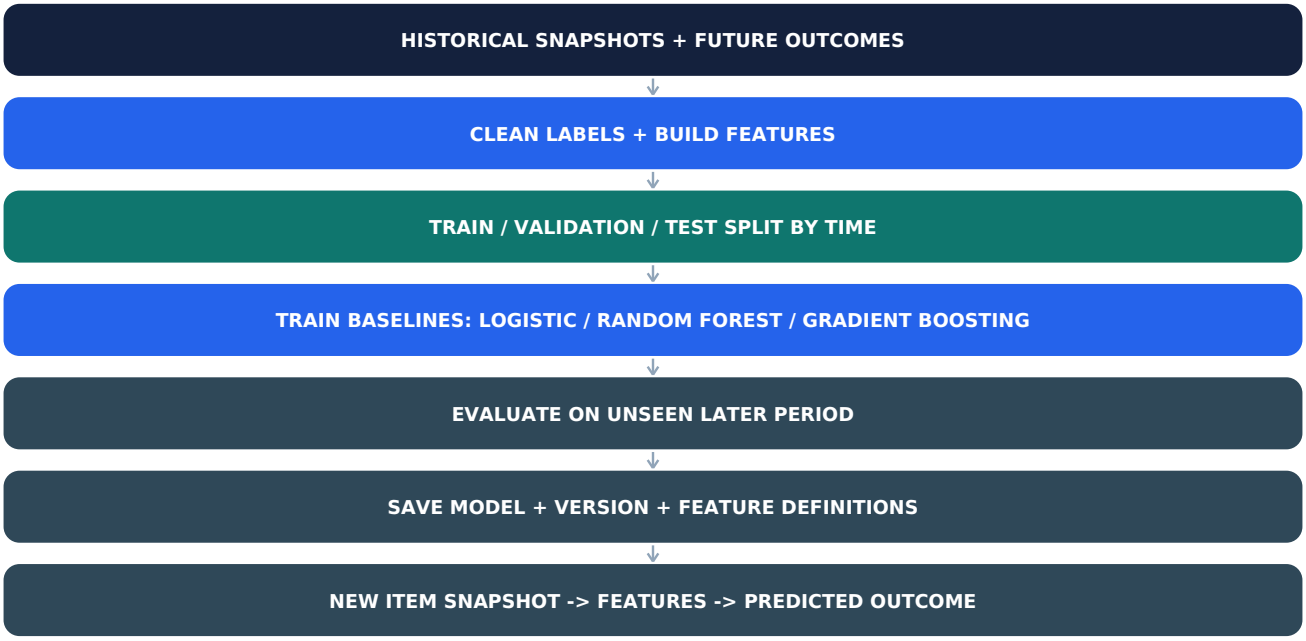
Predict future growth, not vague “quality”

For the first version, predict a defined outcome such as log(1 + citations gained over 180 days) for papers or stars gained over 90 days for repositories. If using high/medium/low labels, define the thresholds from historical data and document them. Treat this as predicted future impact/adoption, not an objective measure of intrinsic scientific quality.

Step 2 - Build the feature vector

Feature group	Possible features	Important rule
Content	Domain, text embedding, paper length, method keywords	Use only data available at prediction time.
Temporal	Age at snapshot, days since last update, recent growth rate	Use snapshots and windows; avoid accidentally using future values.
Popularity/activity	Citation count, stars, forks, contributors, release/commit activity	Normalize for item age and field where appropriate.
LLM-derived	Structured novelty/utility/limitation fields	These are auxiliary signals. Validate their reliability; do not use them as ground truth by default.

Step 3 - Train and use the model



Inference for a new item

- 1. Freeze the record at its prediction timestamp, including the current metadata snapshot.
- 2. Apply the same preprocessing and feature-building code used during training.
- 3. Load the saved model and produce the target estimate or class probability.
- 4. Store the prediction, model version, target horizon, and an explanation of the strongest contributing features.

Example: repository growth prediction

Snapshot features available today	Prediction target	What the user sees
Repository age, stars and forks today, recent star velocity, commits, releases, contributors, domain	Stars gained over the next 90 days	Estimated growth band, predicted value or range, key contributing signals, data coverage and model evaluation summary

Model choice

Start with a simple baseline (for example, predict the same growth as the previous period or use linear/logistic regression). Compare it with Random Forest or gradient boosting. Keep the stronger model only if it improves performance on a later, untouched test period. For citation/star counts, a log transform or a rank-based target can help when a small number of items dominate the distribution.

7. Scores, trends, and explainability

Avoid one mysterious “AI score” that mixes every concept together. Expose a small number of clearly named outputs and explain what each one means.

Output	What it means	How to present it
Domain label	The best category for this item.	Show category, confidence, and allow correction.
Predicted impact/adoption	Estimated future citations, stars gained, or chance of a defined high-growth outcome.	Show target horizon, model version, evaluation quality, and uncertainty.
Signal / priority score	A ranking to help users decide what to inspect first.	Label it as a ranking score, explain its inputs, and allow user-specific filters.
Content quality rubric	Completeness/readability/reproducibility indicators based on explicit criteria.	Use separate criteria and human validation; do not equate it with future popularity.
Trend status	Whether activity is emerging, rising, stable, or declining.	Display historical counts and the time window along with the forecast.

Trend analysis workflow



Hype vs. adoption signal

Compare the growth in research attention (paper count, mentions) with implementation/adoption proxies (active repositories, stars gained, contributors, releases). A large gap can trigger a “high attention, limited adoption evidence” flag. It is a hypothesis-generating signal, not proof that a topic is hype.

Technology radar

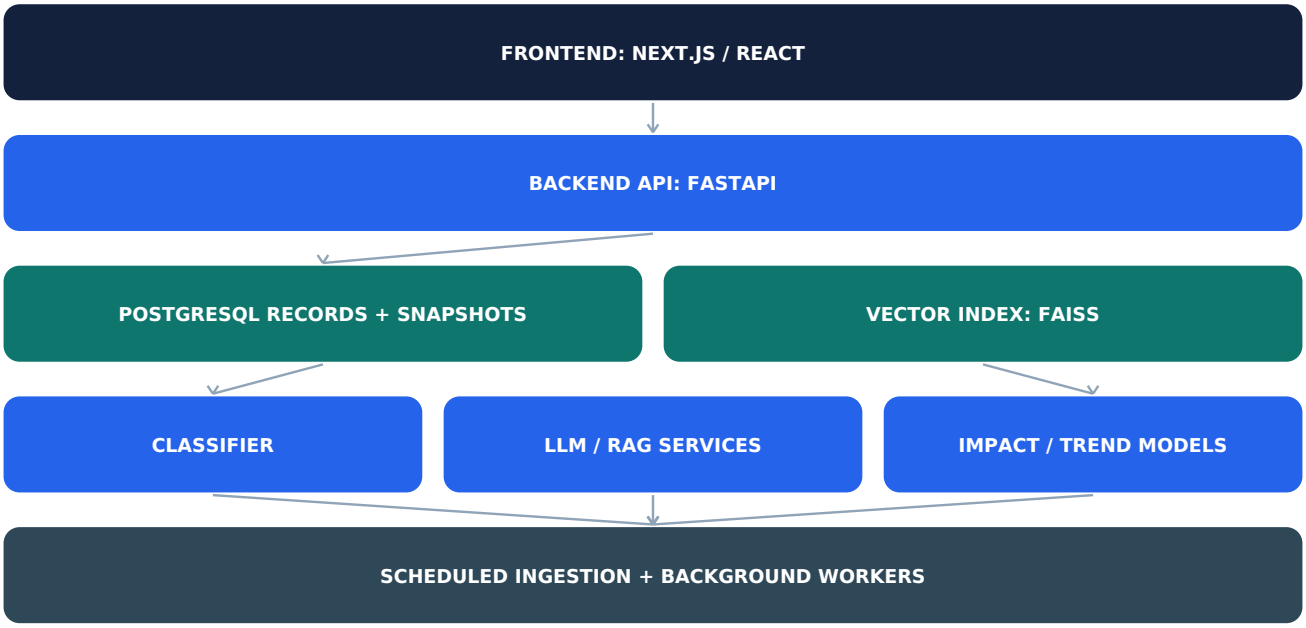
Place topics into Emerging, Rising, Established, or Declining groups based on transparent rules or a validated model. Each topic card should provide the count/time chart, most influential examples, GitHub activity, and a short evidence-backed explanation.

Research opportunities

The LLM can compare recurring limitations across related papers and propose potential gaps. Display these as “candidate research directions” with supporting papers and counter-evidence. Require users to verify them; absence in the collected corpus does not prove a gap is new.

8. Web application and recommended architecture

The expected deliverable is an interactive web application, not just a notebook or model endpoint. Keep ingestion and model jobs separate from the user-facing API so a slow LLM call does not make the dashboard unresponsive.



Suggested pages

Page	Purpose
Dashboard	New items, high-signal items, domain distribution, top movers, recent forecasts.
Explore	Search and filter by date, source, domain, predicted impact, signal score, and trend.
Item detail	Original link, metadata, summary, contributions, limitations, prediction and explanation.
Technology radar	Topic growth charts, relevant papers/repositories, adoption signals, trend status.
Research assistant	Ask questions against the stored corpus; answers include supporting links.
Model evaluation	Classification metrics, forecast errors, calibration and historical prediction outcomes.

Suggested technology stack

Layer	Practical choice
Frontend / charts	Next.js or React; Plotly or a charting library
Backend	Python + FastAPI
Structured storage	PostgreSQL
Embeddings / retrieval	SentenceTransformers + FAISS (or another vector database)
LLM	A hosted model or local Qwen/Llama through Ollama; choose based on latency/cost constraints
ML	Scikit-learn first; compare with gradient boosting where justified
Scheduling / deployment	APScheduler for a prototype; background workers and Docker as the project matures

9. Evaluation, reliability, and pitfalls

The platform should be evaluated as several components. A single “overall accuracy” number cannot tell whether a failure came from collection, classification, LLM extraction, prediction, or retrieval.

Component	How to evaluate
Ingestion	Duplicate rate, missing-field rate, update latency, source/link validity.
Classification	Macro-F1, per-domain precision/recall, confusion matrix, low-confidence review rate.
LLM analysis	Human review of a sample for factual correctness, missing caveats, grounded claims, and source attribution.
RAG assistant	Retrieval relevance, citation correctness, answer support, ability to say “not enough evidence.”
Impact regression	MAE/RMSE and comparison to simple baselines. For skewed outcomes, inspect log-scale and rank quality.
Impact classification	Macro-F1, precision/recall by class, ROC-AUC where appropriate, and calibration.
Trend prediction	Compare with naive baselines such as “next period equals last period”; evaluate across several time windows.
User value	Time to find relevant items, clicks/saves, user-rated usefulness, recommendation relevance.

Critical modeling safeguards

- Prevent data leakage. Never use future citations, future stars, or features measured after the prediction timestamp to train a forecast for that timestamp.
- Split by time. Train on earlier snapshots, tune on later snapshots, and test on an even later period. Random splitting can make future prediction results look falsely strong.
- Use fair comparisons. Citation practices differ by field; older items naturally have more time to accumulate attention. Normalize or stratify by age/domain.
- Keep labels separate from proxies. Popularity is not the same as technical quality. A paper can be high-quality without broad adoption, or popular without being rigorous.
- Calibrate uncertainty. Do not call an arbitrary model output “87% confidence.” Validate probability calibration or present a qualitative confidence band based on data availability.
- Preserve provenance. Store the source URL, retrieval time, model version, and evidence for claims. Handle missing data and API limits explicitly.

Minimum credible ML demonstration

Compare at least one simple baseline with one stronger model on a time-based holdout set. Report the target definition, test period, sample size, relevant metrics, and known limitations. Do not invent results before running the experiment.

10. Implementation roadmap and demo plan

Phase	Build	Definition of done
1. Scope + dataset	Choose sources, domains, prediction horizon, and label definition. Collect a reproducible starter dataset.	A data dictionary exists and sample records can be re-fetched or replayed.
2. Data pipeline	Implement API clients, scheduling, validation, deduplication, snapshots, PostgreSQL storage.	New records can be ingested repeatedly without duplicate buildup.
3. Classification	Label a starter set and train an embedding-based classifier; compare an LLM baseline.	Metrics and per-class errors are available; uncertain records are handled.
4. LLM / RAG	Structured extraction, evidence links, embeddings, search, source-grounded Q&A.;	Summaries are viewable and claims can be traced to source text/URLs.
5. Prediction	Build temporal features, train baseline and stronger model, evaluate on a later period.	Prediction outputs are reproducible and model quality is documented.
6. Web app	Dashboard, filters, item detail, trends, assistant, evaluation page.	A judge can discover an item and inspect evidence/prediction in a coherent workflow.
7. Polish + demo	Add scheduled refresh, explainability, error handling, prediction history, deployment.	A complete end-to-end demo works without manual database edits.

Suggested “killer demo”

- Click Scan latest sources or open the most recent completed scan. Show counts collected and the timestamp of the run.
- Open a high-signal paper. Show its domain, evidence-linked summary, limitations, matched repository, and predicted future outcome.
- Open the model explanation to show which features contributed and what prediction horizon was used.
- Open the Technology Radar and explain one growth claim using historical monthly counts.
- Ask the research assistant a question. Show citations to retrieved papers and repositories, then open one of the sources.
- Finish on Model Evaluation to show test-period results and at least one example where the model was wrong.

Team workstreams

Workstream	Responsibilities
Data engineering	Source adapters, scheduled jobs, normalization, deduplication, temporal snapshots.
ML / evaluation	Labels, domain classifier, features, prediction model, leakage checks, metrics.
LLM / retrieval	Prompt/schema, evidence extraction, embeddings, RAG, quality checks.
Frontend / backend	API, database integration, dashboard, detail pages, filters, charts.
Integration / QA	End-to-end workflow, error handling, demo script, documentation, deployment.

Final recommendation

Build the reliable core first: collect -> classify -> analyze -> predict -> display -> evaluate. Then add the most distinctive extras: explainable signal ranking, technology radar, evidence-grounded assistant, and research opportunity detection. A smaller system with defensible predictions is stronger than a large system with unvalidated scores.

One-line pitch

TechPulse AI turns the flood of technical research and open-source activity into evidence-backed signals, interpretable forecasts, and practical next steps through an autonomous web platform.